# System Information:

- Operating System: Arch Linux (Omarchy)
- Language: Python 3.14.0

# Instructions:

1. Download zip folder
2. Unzip
3. Open terminal in the **PARENT** directory of the project – if the project was downloaded to your downloads folder, open the terminal at Downloads/ and run the command in step 4 from there
4. Use “python -m aes_m15219531.build.aes_first_round”

# Screenshots:

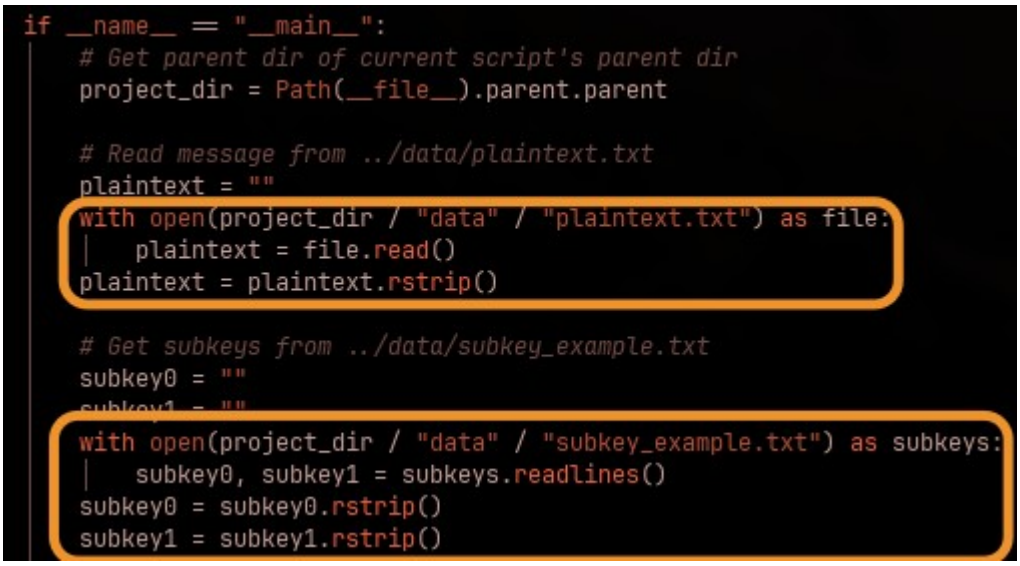


*Figure 1: Reading **plaintext.tx**t and **subkey_example.txt** from the **data/** directory in **build/aes_first_round.py***

```python
def aes_first_round(plaintext_input, subkey0_input, subkey1_input):
    # Convert to ASCII (ord), then to hexadecimal (hex), and then to hex literal (int)
    text = [int(hex(ord(character)), 16) for character in plaintext_input]

    # Translate subkeys into usable hex literals
    subkey0 = _string_to_hex(subkey0_input)
    subkey1 = _string_to_hex(subkey1_input)

    # Create 2D array for text, subkey0, and subkey1
    text = _swap_cols_rows(_generate_4x4_matrix(text))
    subkey0 = _swap_cols_rows(_generate_4x4_matrix(subkey0))
    subkey1 = _swap_cols_rows(_generate_4x4_matrix(subkey1))

    # Perform AES calculations
    text = _add_key(text, subkey0)
    text = _sub_bytes(text)
    text = _shift_rows(text)
    text = _mix_columns(text)
    text = _add_key(text, subkey1)

    return text
```

*Figure 2:* ***build/aes_first_round.py*** *has the function to call the required AddKey (before round 1), SubBytes, ShiftRows, MixColumns, and AddKey (during round 1)*

```python
def _add_key(text, subkey):     ● "_add_key" is not accessed
    """
    text: 4x4 matrix to perform AddKey on
    subkey: subkey to use in the AddKey calculation

    returns: list of len(text)
    """
    flattened_text = [item for sublist in text for item in sublist]
    flattened_subkey = [item for sublist in subkey for item in sublist]

    output_array = []
    count = 0
    while count < len(flattened_text):
        output_array.append(flattened_text[count] ^ flattened_subkey[count])
        count += 1

    output_array = _generate_4x4_matrix(output_array)

    print(f"AddKey: {_hex_convert(output_array)}")
    return output_array
```

*Figure 3:* ***src/aes_functions.py*** *has the AddKey function, where the 2D arrays are flattened and XOR'ed against eachother with the ^ operator*

```python
def _sub_bytes(text):     ● "_sub_bytes" is not accessed
    """
    text: 4x4 matrix of ints

    returns: list of ints
    """
    sbox = [

    flattened = [item for sublist in text for item in sublist]

    text_strings = []
    for num in flattened:
        text_strings.append(str(hex(num)))

    sbox_key = []
    for num in text_strings:
        # Handle cases like 0x0, 0xe
        if len(num) == 3:
            sbox_key.append([0, int(num[-1], 16)])

        # Handle cases like 0x43, 0xf3
        else:
            sbox_key.append([int(num[-2], 16), int(num[-1], 16)])

    output_array = []
    for key in sbox_key:
        output_array.append(sbox[key[0]][key[1]])

    output_array = _generate_4x4_matrix(output_array)

    print(f"SubBytes: {_hex_convert(output_array)}")
    return output_array
```

*Figure 4:* ***src/aes_functions.py*** *has the SubBytes function, including the S-Box table (collapsed for readability), table lookup, and 2D matrix reconstruction and output*

```python
def _shift_rows(text):     ● "_shift_rows" is not accessed
    """
    text: 4x4 matrix

    returns: 4x4 matrix
    """
    output_array = []

    index = 0
    while index < len(text):
        output_array += [text[index][index:] + text[index][:index]]
        index += 1

    print(f"ShiftRows: {_hex_convert(output_array)}")
    return output_array
```

*Figure 5: **src/aes_functions.py** has the ShiftRows function that adjusts row 0 by 0 places, row 1 left once, row 2 left twice, and row 3 left 3 times*

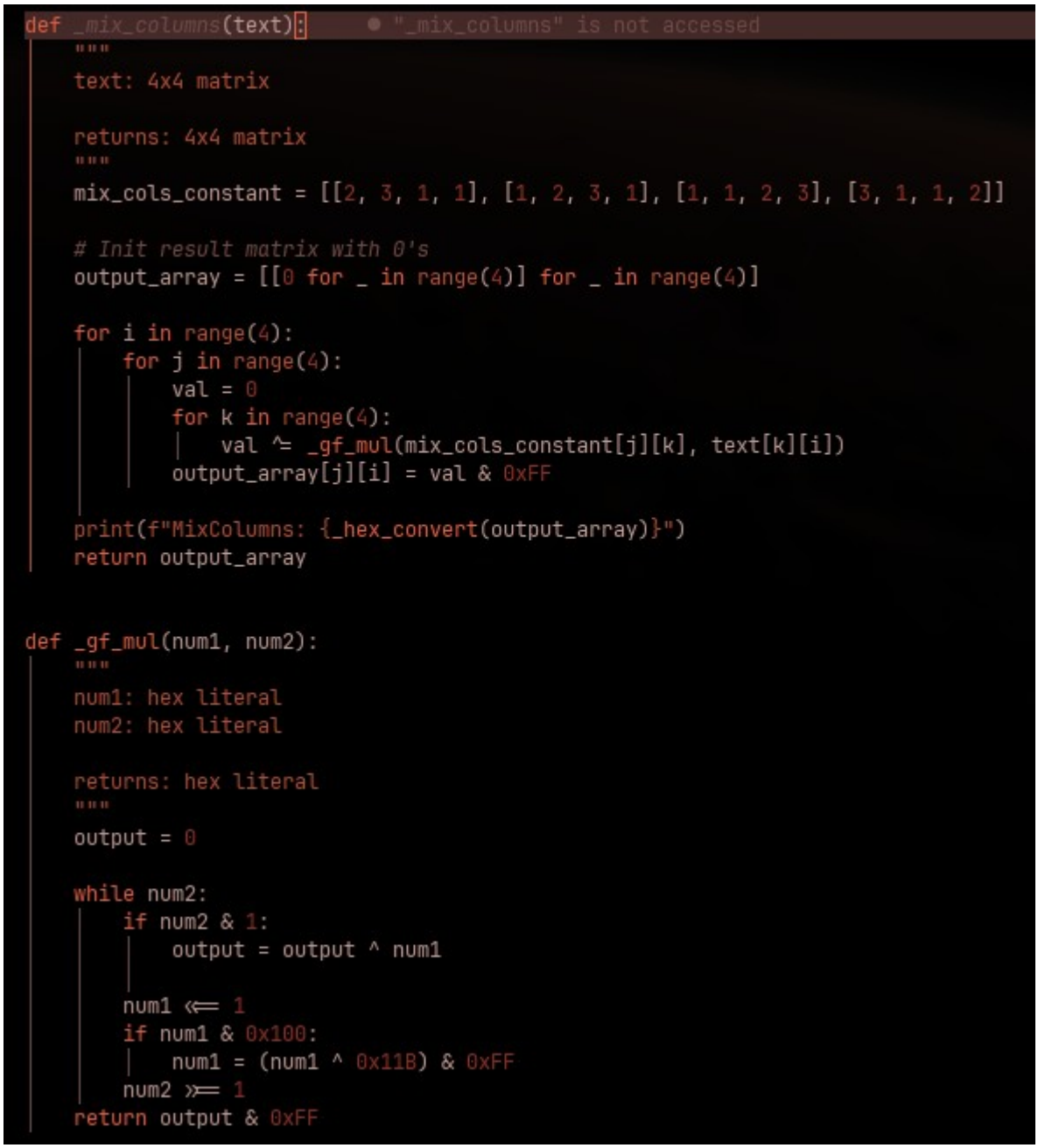


*Figure 6: **src/aes_functions.py** has the MixColumns function (and its helper) that performs the necessary MixColumns calculation on the text matrix*